

Support Ticket Router

Individual Hackathon Challenge · 2 Hours Maximum

Instructions

1. This hackathon is conducted individually. Team participation is not allowed.
2. You may use AI tools such as ChatGPT, Gemini, Claude, GitHub Copilot, or Cursor to assist with problem-solving and development.
3. Use AI effectively, but make sure you understand and can explain every part of your solution.
4. This task is also testing your AI integration skills, so please set up a real AI provider properly rather than relying on regex or other rule-based workarounds. Not having a paid API key is not a valid excuse. Free API keys are readily available from providers such as OpenRouter, Groq, and Google AI Studio.
5. During evaluation, you may be asked to explain your approach, the AI prompts you used, your architecture, and your implementation decisions.
6. Copying another participant's solution, or submitting work that isn't your own, is strictly prohibited and may lead to disqualification.
7. Your submission must be original and developed during the hackathon.
8. Internet access is permitted for documentation, learning resources, and AI tools.
9. Manage your time well and submit your solution before the deadline.
10. Make sure your code runs, and include a short README covering the points below.

Your README should explain:

- A. AI tools used, if any.
- B. How to run the project.

Evaluation Criteria

- Problem-solving approach.
- Code quality and structure.
- Technical understanding.
- A working solution, and your ability to explain the implementation.

The Scenario

Imagine you're working at Zomato. Every day, thousands of customers reach out to support with messages like these:

- "My order never arrived."
- "I was charged twice."
- "I can't log in."
- "Where is my refund?"

Right now, a support agent reads every ticket before routing it to the right team. That takes time, and urgent issues can end up waiting longer than they should.

Your job is to build a small service that handles this first step automatically.

What Your System Should Do

When a new support ticket comes in, your system should:

- Understand what the issue is.
- Decide how important it is.
- Suggest a reply, unless the message is abusive.
- Save the ticket so agents can search and review it later.

APIs to Build

No frontend is required. You'll build four endpoints:

POST	/tickets	Create and classify a new ticket.
GET	/tickets/{id}	Retrieve a single ticket.
GET	/tickets	List tickets, with filters for category and priority.
POST	/tickets/{id}/feedback	Record whether the AI's classification was correct.

Important Rules

- If the customer sounds angry, increase the priority by one level.
- If the message is abusive, don't generate a suggested reply.
- Never send the customer's email to the AI.
- The same ticket should always receive the same priority.
- Enterprise customers with three or more previous tickets should always be classified as High priority or above.

Use the AI to understand the ticket (category, sentiment, tags, and reply). Use your own code to apply these business rules consistently.

Write-Up: DECISIONS.md

Ship a DECISIONS.md file alongside your code. Bullet points are fine. We care about depth, not length.

Write this file yourself, in your own words. Do not generate it with AI.

Please cover:

- The stack you chose, and why.
- How you structured the solution, specifically what you let the LLM decide versus what you kept in code, and why you drew the line where you did.
- The trade-offs you made, and what you deliberately skipped for time.
- What you'd add before taking this to production.

If You Have Time (Bonus)

Not required, and not worth sacrificing the core experience for. But if you're ahead of schedule, consider:

- Deploying it live on a free host, such as Render, Railway, or Fly, and sharing the URL.
 - Containerizing it with a Dockerfile or docker compose up.
 - Using an ORM cleanly, with models and migrations.
 - Anything else thoughtful, such as indexes where they help, duplicate detection, or idempotency.
-

We're not looking for all of it in two hours. We're looking for a working core, sharp judgment about the rules, and a clear head about what you chose to leave out.